Self-Healing Microservice Mesh

A small, fully working **self-healing microservice system written in Go**. A group of deliberately fragile microservices is watched by a Go **control plane** that detects failures with goroutine-based health checks, automatically restarts broken services, and re-routes traffic to healthy replicas. It is a mini Kubernetes-style control loop with a live web dashboard.

Phase-II Capstone project by Tripti (2301010278), Rizpah (2301010219), Raghuveer (2301010268), Ayush (2301010253), Aman (2301010277) and Sarthak (2301010216).

1. Objectives

- Detect service failures (crash, hang, slow responses, error responses) within a few seconds.
- Recover automatically, with no manual steps (restart with exponential backoff).
- Keep users unaffected while a replica is down (health-aware routing with retry).
- Make the system observable: live dashboard, event timeline, MTTD and MTTR metrics.
- Show Go's concurrency primitives (goroutines, mutexes, atomics, `context` and tickers) in a real operational story.

2. Key Features

Feature	What it does
Fragile services	3 services (A, B, C), 2 replicas each, each can be crashed, hung, slowed or made to return errors on demand
Health probing	One goroutine per replica probes `/health` every second with an 800 ms timeout
Health state machine	healthy, suspect (1 miss), unhealthy (3 misses), restarting, starting, back to healthy (2 passes)
Auto-restart	Supervisor kills and restarts the process with exponential backoff (1s, 2s, 4s … max 16s)
Smart routing	Round-robin only over healthy replicas, retry on a second replica if the first fails
Reconciler	Declarative desired state (replicas per service); scale up or down from the UI and the loop fixes the difference
Chaos controls	Crash, Hang, Slow, Errors and Heal buttons on every replica; optional `-random-chaos` flag
Live dashboard	KPIs, per-service success-rate sparkline, replica cards, event timeline, MTTD / MTTR

3. Technologies Used

- **Go 1.22+**, standard library only (`net/http`, `os/exec`, `context`, `sync`, `sync/atomic`, `embed`). No external dependencies.
- **HTML, CSS and vanilla JavaScript** for the dashboard (embedded into the Go binary with `go:embed`).
- JSON REST API between the dashboard and the control plane.

4. Architecture

```
 Client / Traffic generator
          |
   [ Router: round-robin over healthy replicas, retry once ]
      |            |            |
  Service A     Service B     Service C        (2 replicas each,
separate OS processes)
      ^            ^            ^
      +---- Control plane (Go) ---+
            Prober      one goroutine per replica, GET /health every
1s
            Reconciler  every 500ms: desired vs actual state, restarts
/ scales
            Supervisor  starts and kills processes, exponential
backoff
            API + UI    /api/state, /api/chaos, /api/scale,
/api/traffic, dashboard on /
```

Failure timeline (hang fault): t=0 replica hangs, t=1s first probe fails (suspect), t=3s third failure (unhealthy, removed from routing), t=4s supervisor restarts it, t=6s two good probes (healthy, back in rotation).

5. Implementation Overview

File	Purpose
`cmd/service/main.go`	The fragile microservice: `/health`, `/work`, `/chaos/<mode>`
`cmd/controlplane/main.go`	Prober, reconciler, supervisor, router, REST API
`cmd/controlplane/web/index.html`	Dashboard UI (served from the binary)

Key design points: probes run concurrently in goroutines; all shared state is protected by a `sync.RWMutex`; request counters use `sync/atomic`; probes use HTTP client timeouts so a hung service cannot block detection; the reconciler is level-triggered (it acts on current state, not individual events), so it is safe to repeat.

6. How to Run

Requirements: **Go 1.22 or newer** (go.dev/dl). Ports **8080** and **9100-9120** must be free.

```
git clone https://github.com/<your-username>/self-healing-mesh.git
cd self-healing-mesh
go run ./cmd/controlplane
```

Open **http://localhost:8080**. The service binary is built automatically on first start. Stop with `Ctrl+C` (all service processes are stopped too).

Optional flags: `-addr :8080`, `-replicas 2`, `-random-chaos` (services break themselves randomly). Send requests yourself: `curl http://localhost:8080/svc/a/work`

7. Demo Walkthrough

1. Click **Start traffic** (5 requests/second per service).
2. Press **Crash** on `a-1`: it is removed from routing instantly and restarted; success rate stays 100%.
3. Press **Hang** on `b-1`: it turns amber, then red after 3 failed probes, restarts, and returns to green.
4. Press **Errors** on `c-1` and watch the suspect state, then **Heal** to clear the fault.
5. Press **+** on a service to change the desired state; the reconciler starts a new replica.

8. UI Screenshots

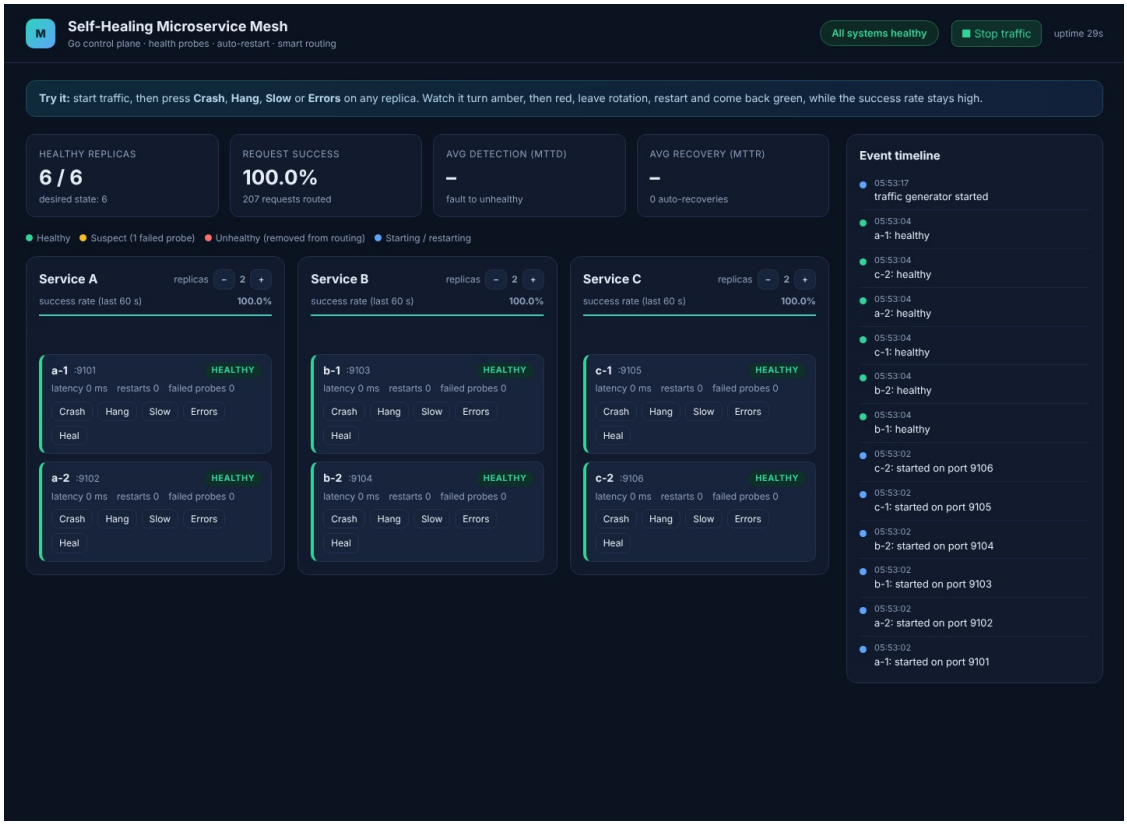


Figure 1: Dashboard with all replicas healthy and traffic running

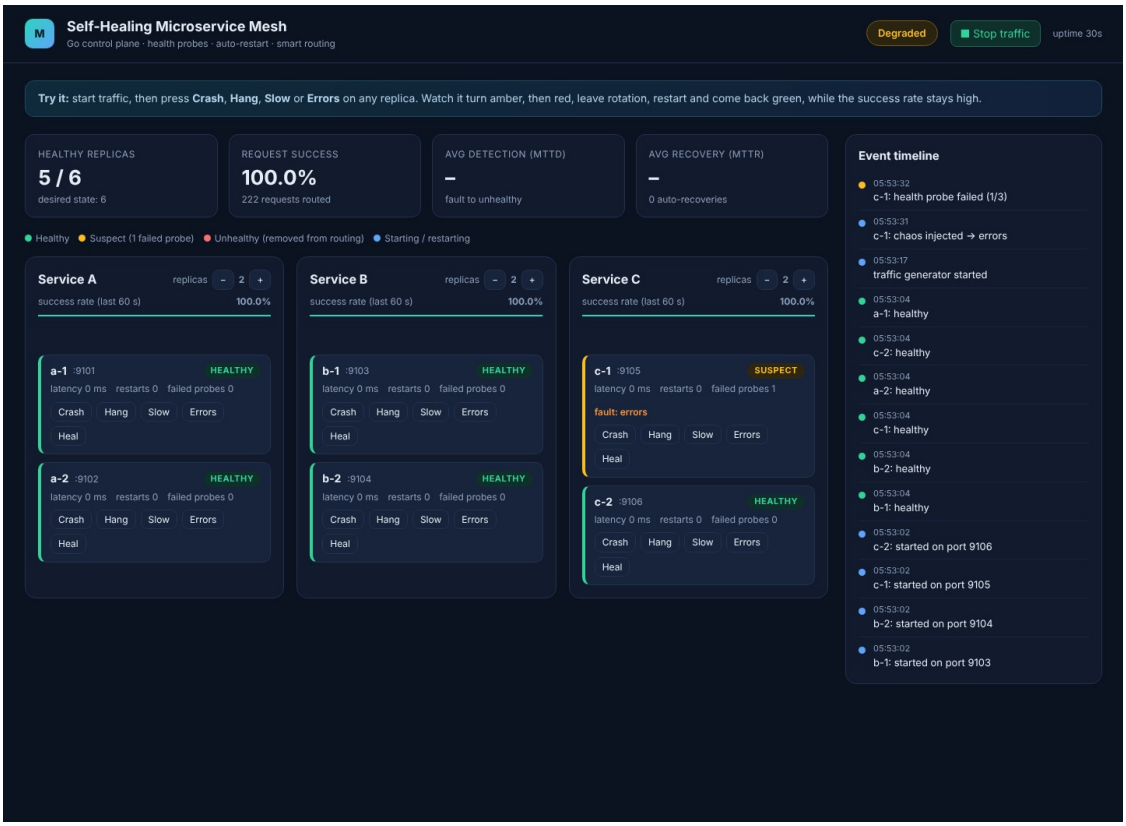


Figure 2: A replica returns errors and becomes Suspect (amber)

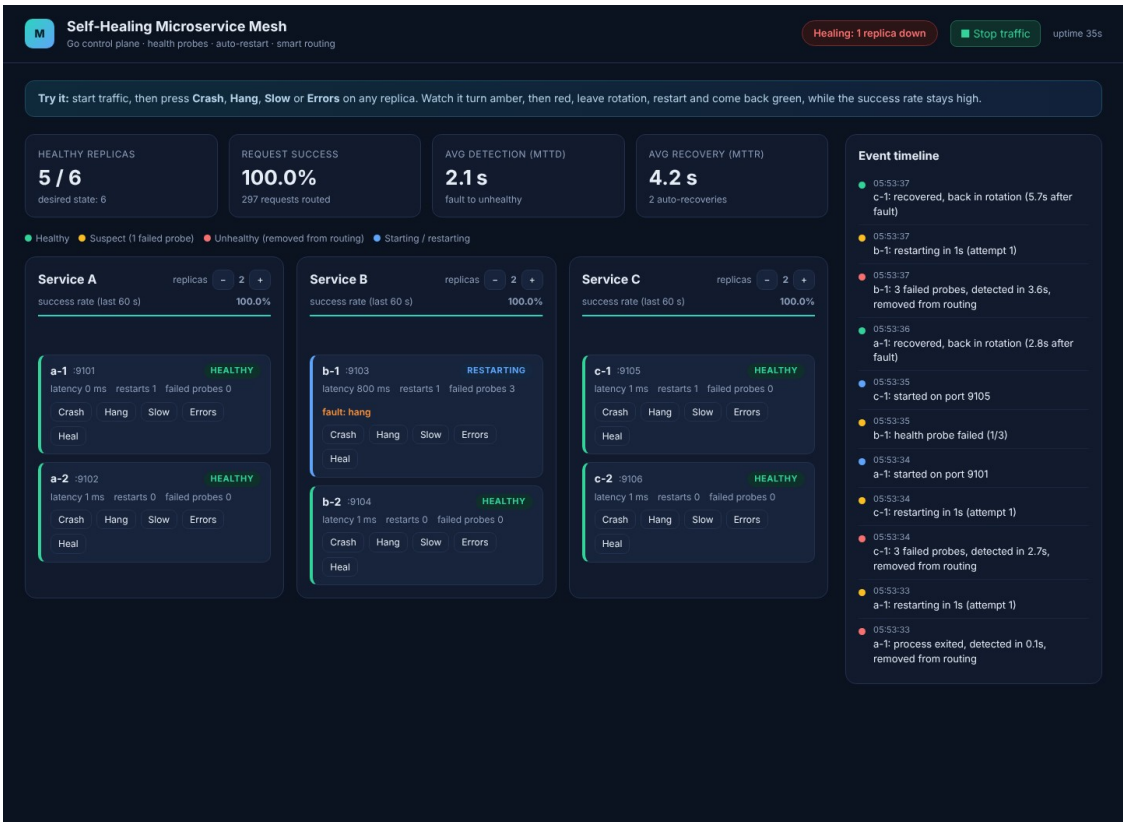


Figure 3: Failures detected, replicas removed from routing and restarting

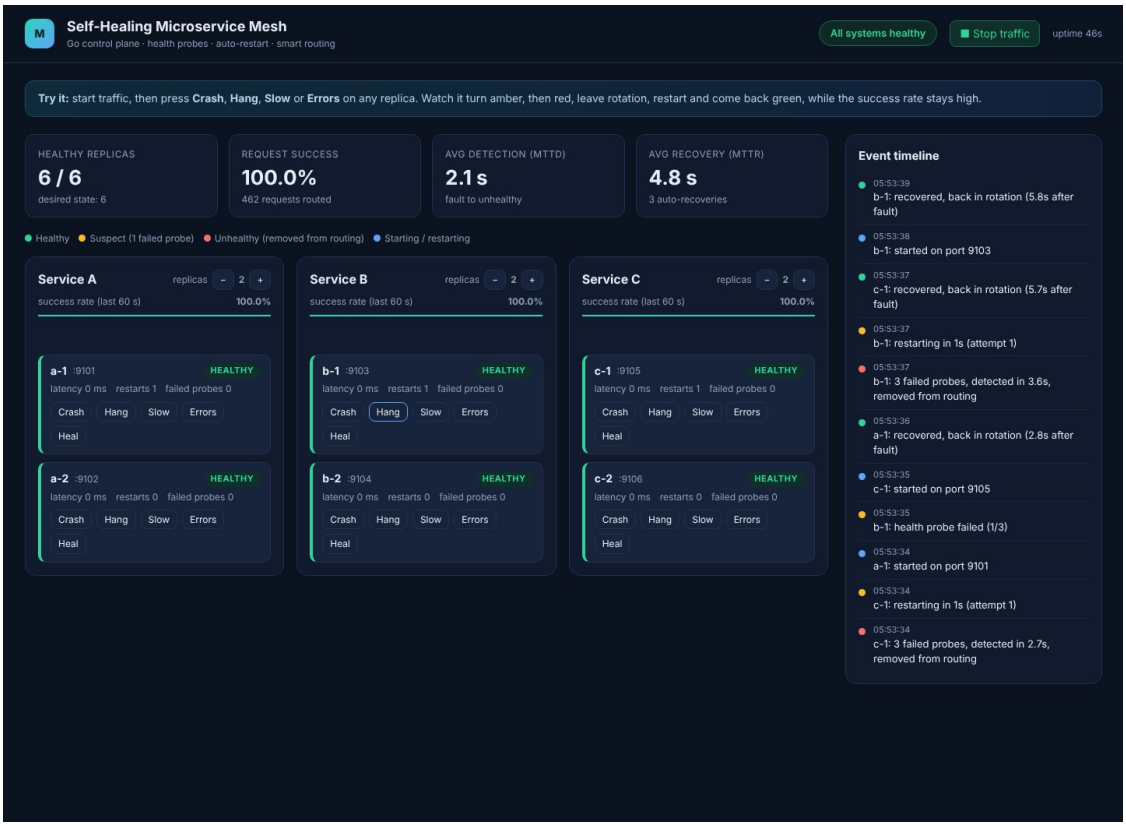


Figure 4: All replicas healed automatically; MTTD and MTTR shown

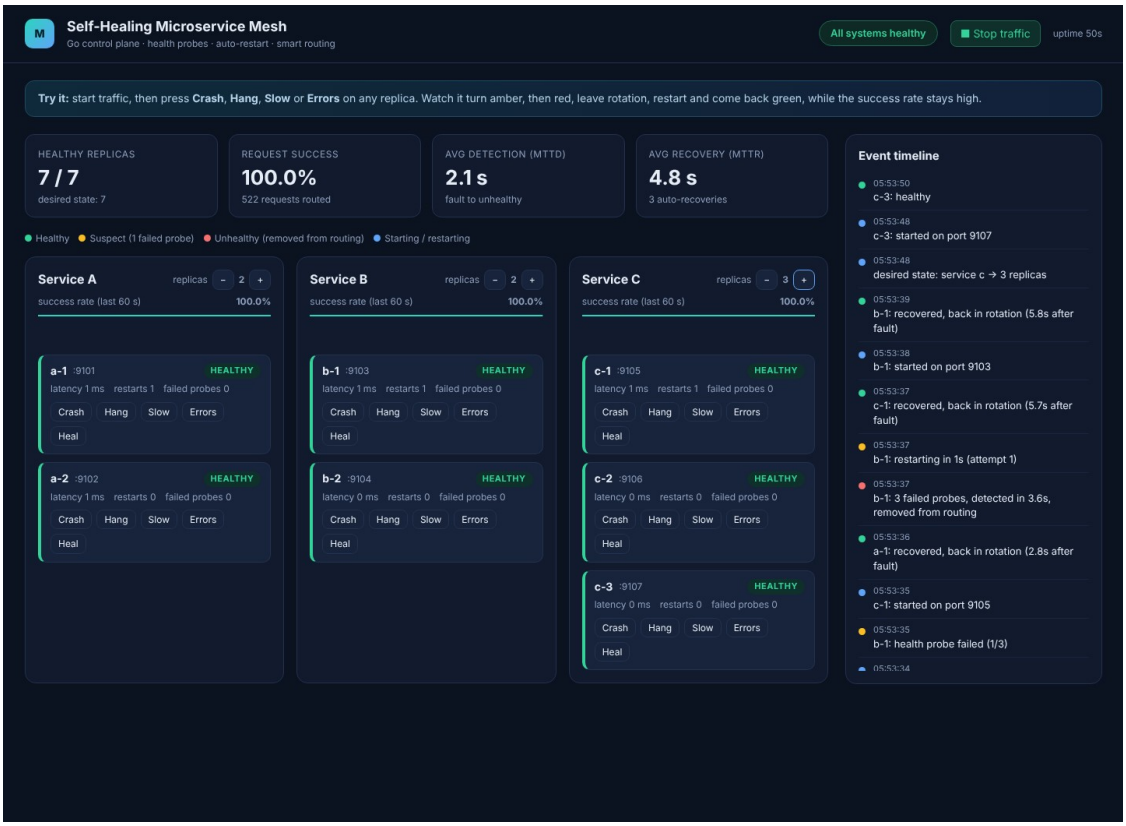


Figure 5: Desired state changed from the UI; reconciler starts a new replica

9. Test Results (local run)

Observed with 2 replicas per service and traffic running: process crash detected in about 0.1 s, hang or error faults detected in about 2 to 4 s, replica back in rotation about 3 to 6 s after the fault, and **100% request success** (297 of 297 requests) while replicas were crashing and hanging.

10. Limitations and Future Scope

Single control-plane instance (no leader election), local processes instead of containers, GET requests only through the router. Future work: Raft leader election, Docker or container backend, circuit breakers, Prometheus metrics, auto-scaling by load.